

UNIVERSITÀ DI PISA

Decentralised Lending Service

Marco Pernisco / Matricola 683674 P2P Systems and Blockchains Lab 2025/26

1. Architettura e protocollo

Il servizio raccoglie ETH in un fondo comune e affida ai partecipanti il voto sui prestiti. Il **contributore** deposita, vota, preleva capitale disponibile e riscuote interessi o compensazioni. Il **richiedente** propone le condizioni, risolve la proposta e, se approvata, riceve e rimborsa ETH. Un account può avere entrambi i ruoli.

Componenti on-chain. `FundingPoolProxy` conserva ETH e stato; `FundingPool` fornisce la logica UUPS, aggiornabile solo dal proprietario. I test verificano che lo stato sia preservato. Ogni prestito approvato crea un `LoanContract` distinto, con debitore, condizioni e rimborsi propri. Aggiornare il pool non cambia il bytecode dei prestiti già creati. `BitcoinOracle` memorizza saldi in satoshi ed emette richieste di aggiornamento a pagamento; il pool ne legge il saldo quando risolve una proposta.

Componenti off-chain. L'oracolo Python fidato elabora i primi **131.000 blocchi Bitcoin mainnet**, uno alla volta, ricostruendo l'indice UTXO **in memoria**, poi risponde alle richieste Ethereum. Saldi Bitcoin e UTXO non sono salvati su disco: ogni riavvio ripete la scansione. Il voter segue gli eventi di proposta e vota sempre YES, anche con capitale tutto bloccato. SQLite conserva eventi e transazioni firmate, non saldi Bitcoin, per riprendere dopo errori RPC temporanei.

Modello di fiducia. Non si prova la proprietà dei BTC e non li si blocca. La liquidità storica Bitcoin è valutata al cambio fisso **1 BTC = 30 ETH**. Il parser controlla collegamento dei blocchi, hash e radici Merkle, esclude la ricompensa non spendibile del genesis e rifiuta dati incompleti; non è un nodo che verifica l'intero consenso Bitcoin.

Votazione e finanziamento

I depositi devono superare **100.000 wei**. Anche chi ha capitale bloccato può votare, ma la risoluzione pesa il voto sul **saldo disponibile corrente**. L'astensione vale NO; la parità respinge. Solo il richiedente originario può risolvere, dopo almeno **12 blocchi**. Si può votare fino alla risoluzione: `votingDeadlineBlock` indica il primo blocco utile, non la fine del voto. Tutte le durate sono differenze di altezza. La proposta specifica capitale, tasso di interesse (1–100), durata e indirizzo BTC.

ETH disponibili o liquidità Bitcoin insufficienti chiudono e respingono la proposta. L'approvazione chiude il voto e blocca quote proporzionali del capitale disponibile dei contributori. I resti della divisione intera riducono il **capitale effettivamente erogato**, senza assegnarli arbitrariamente. La percentuale di collateral resta fissata per quel prestito alla creazione.

Rimborso, interessi e compensazione

Il capitale è **disponibile** o **bloccato**; interessi da riscuotere e riserva di compensazione sono saldi separati, non prestabili. Solo il debitore originario può rimborsare, anche dopo il default. Ogni rata copre capitale e interessi in proporzione al debito totale; si può saldare tutto in un pagamento.

Il **rimborso del capitale** sblocca fondi nel pool, non li invia al wallet. Si restituisce prima a chi aveva la quota *iniziale* maggiore; a parità, vince l'indirizzo numericamente maggiore. L'ordine si calcola una sola volta. Gli **interessi** sono divisi usando il collateral fisso del prestito: una parte entra nella riserva; il resto è un guadagno proporzionale alle quote iniziali, riscosso con `claimInterestGain()`. I calcoli cumulativi rendono coerenti gli arrotondamenti di capitale, interessi e collateral tra le rate. Eccedenze oltre il debito totale e resti della distribuzione degli interessi vanno alla riserva. A pagina 4 è riportato un esempio numerico.

Successo e default. Il saldo completo entro il blocco di scadenza ha successo; un prestito scaduto e non saldato è segnato fallito da una transazione, non dal solo passare del tempo. Il fallimento si conta una volta e non è annullato da rimborsi tardivi. Il collateral globale parte dal **50%**, resta tra **1 e 100%**, scende di cinque punti per successo e sale di cinque per fallimento; i prestiti esistenti mantengono la propria percentuale.

La **compensazione** copre solo capitale non restituito, nei limiti della riserva. Se vuota, la richiesta registra comunque il default, paga zero e conserva il credito per futuri reintegri. Quanto pagato riduce di pari importo il diritto a recuperare capitale dal debitore, senza cambiare i pesi degli interessi. Solo il proprietario può **terminare** il servizio, senza prestiti attivi; prelievi, interessi e recuperi restano possibili.

2. Sicurezza e limiti operativi

Reentrancy: modifica e attacco eseguibile

`FundingPoolVulnerable`, usato solo nei test, eredita dal pool reale e sostituisce il prelievo con scritture di saldi memorizzati **dopo** il trasferimento esterno:

```
uint256 beforePosition = totalContributed[msg.sender];
uint256 beforePool = totalUnlockedFunds;
(bool sent, ) = payable(msg.sender).call{value: amount}("");
if (!sent) revert TransferFailed();
totalContributed[msg.sender] = beforePosition - amount;
totalUnlockedFunds = beforePool - amount;
```

`ReentrancyAttacker` richiama ricorsivamente `withdraw(amount)` dalla funzione di ricezione. In `test/reentrancy.test.ts`, un deposito di 1 ETH dell'attaccante permette di sottrarre i 10 ETH del contributore onesto dal proxy vulnerabile. Lo stesso attaccante non ruba quei 10 ETH dal proxy normale, che aggiorna la contabilità prima del trasferimento. Una semplice sottrazione dal saldo corrente dopo la chiamata potrebbe invece causare un revert per underflow in Solidity 0.8; le assegnazioni di valori ormai obsoleti rendono l'esempio davvero sfruttabile. Le varianti di test non sono mai distribuite dallo setup del servizio. Queste prove non garantiscono la sicurezza generale.

Guadagno scorretto senza reentrancy

Un contributore di maggioranza con **6 ETH** può approvare un prestito da **10 ETH** a un debitore complice, finanziato anche dai **4 ETH** di un partecipante onesto, e poi non rimborsarlo. Il gruppo colluso guadagna **4 ETH netti**, non 10, prima di gas e altri flussi. Può indicare qualsiasi indirizzo BTC abbastanza ricco, perché non si prova la proprietà. Anche la priorità dei rimborsi favorisce le quote grandi: con quote 6/4, un rimborso di capitale pari a 6 ripristina la posizione maggiore prima che il contributore minore recuperi capitale. Sono conseguenze delle regole economiche del protocollo, non di una vulnerabilità reentrancy.

Limiti. Le posizioni vuote vengono rimosse, ma il lavoro cresce con il numero di contributori e deve rientrare nel limite di gas del blocco. I checkpoint rilevano la sostituzione della chain; i conflitti di nonce restano ispezionabili. Il recupero dalle riorganizzazioni non è automatico.

3. Esecuzione del servizio

Eseguire i comandi dalla **radice del progetto**, salvo diversa indicazione. Servono una versione Node pari supportata (22.13.0+), Python 3.10+ e **Geth 1.13.14**, client Clique precedente a PoS compatibile con il genesis della rete privata.

1. Installare le dipendenze e validare i dati Bitcoin

```
./setup.sh
./.venv/bin/python python/oracle/check_data.py --data-dir bitcoin --summary
```

Lo setup installa le dipendenze Node bloccate e i pacchetti Python, crea `.venv`, compila i contratti e scarica Geth. Installazione e download del compilatore non già in cache richiedono internet; saltare lo setup se già completato. Fornire file mainnet `blk*.dat` decodificati dal genesis all'altezza 130.999. Il controllo offline non richiede chiavi o Geth: il successo riporta `valid: true`, **131.000 blocchi**, altezza **130.999** e indirizzi con fondi. Non sostituire dati mancanti o incompleti con saldi fittizi.

2. Avviare la blockchain privata / terminale 1

```
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore --networkid=202526 \
  --http --http.addr=127.0.0.1 --http.port=8545 --http.api=eth,net,web3 \
  --mine --miner.gasprice=100000000 \
  --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

Lasciare il nodo attivo.

3. Creare gli account e distribuire / terminale 2, radice del progetto

```
./.venv/bin/python python/setup.py
```

L'account prefunded fornito **trasferisce solo ETH** a tre account appena generati. Il nuovo deployer crea oracolo, implementazione e proxy inizializzato. Lo setup ricompila il bytecode ordinario, verifica chain ID, ricevute, codice e inizializzazione, poi scrive configurazioni protette con `DEPLOYMENT_BLOCK`. Ripeterlo crea una distribuzione separata.

4. Avviare oracolo e voter / terminali separati

```
./.venv/bin/python python/oracle/oracle_service.py
./.venv/bin/python python/voter/auto_voter.py
```

Attendere **ORACLE READY**: l'oracolo deve terminare la scansione Bitcoin iniziale in memoria. Non c'è un'attesa fissa da configurare; ogni riavvio ripete la scansione. Lasciare attivi entrambi i servizi durante la demo e conservare le code SQLite dopo errori RPC temporanei: sono separate dall'indice Bitcoin.

5. Simulare rimborso e default / altro terminale, radice del progetto

```
./.venv/bin/python python/simulate.py \
  --btc-address 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa --principal 1
```

Questo indirizzo ha **4.000.000 satoshi** nello snapshot validato, sufficienti per 1 ETH al cambio fisso. Lo scenario A rimborsa 1 ETH al 5% in due rate e riscuote il guadagno; B lascia insoluti 0,5 ETH, richiede compensazione e riprova a riserva vuota. Entrambe le proposte attendono voti YES e 12 blocchi reali. Saldi, stati e controlli contabili sono stampati dopo le modifiche significative.

Tempi e ripresa. I blocchi Clique richiedono circa 10 secondi: prevedere alcuni minuti per le due votazioni. `-timeout` imposta i secondi massimi *per ogni fase asincrona* (300 predefiniti). In caso di timeout, controllare nodo e log dei servizi prima di riprovare; non eliminare chain, chiavi, configurazioni o code. La contabilità è spiegata a pagina 4; riusare una distribuzione può produrre numeri diversi.

4. Simulazione: rimborso e default

L'esecuzione locale osservata usa `-principal 1`. L'oracolo risponde con **4.000.000 satoshi = 0,04 BTC**, pari a **1,2 ETH** di capacità al cambio di 30 ETH/BTC. I BTC non sono trasferiti né bloccati. Il contributore deposita **50 ETH**: il wallet passa da 100 a circa 50 ETH dopo il gas, mentre il pool contiene 50 ETH di capitale disponibile.

Primo prestito: 1 ETH, interesse al 5%, due rate

Dopo il voto YES e 12 blocchi, l'approvazione crea e finanzia un contratto dedicato. Il pool contiene 49 ETH; il contributore ha **49 disponibili + 1 bloccato**. Il debito è **1,05 ETH**. Con **collateral al 50%**, ogni rata da 0,525 ETH restituisce 0,5 ETH di capitale e paga 0,025 ETH di interessi, divisi a metà tra riserva e guadagno del contributore:

Destinazione	Ogni rata (ETH)	Totale (ETH)
Capitale ripristinato come disponibile	0,5000	1,0000
Interessi accreditati alla riserva	0,0125	0,0250
Interessi accreditati come guadagno	0,0125	0,0250
Pagamento del debitore	0,5250	1,0500

Il prestito diventa `Repaid`: il capitale disponibile torna a 50 ETH e quello bloccato a zero. Il contributore riscuote 0,025 ETH di guadagno; nel pool restano **50 ETH di capitale + 0,025 ETH di riserva**. Il collateral globale scende dal 50 al **45%**; quello del primo prestito resta al 50%.

Secondo prestito: la scadenza non esegue transazioni

Una nuova proposta riceve YES ed è risolta dopo altri 12 blocchi. Eroga **0,5 ETH** al 10%, con collateral fisso al **45%**, e non viene rimborsata intenzionalmente. Il finanziamento avviene al blocco **328**, la scadenza è **331**. Al 332 il prestito è scaduto ma resta memorizzato come `Funded`: il solo avanzare dell'altezza non esegue codice. La transazione di compensazione al blocco **333** registra `Defaulted`, azzera i prestiti attivi, conta un fallimento e riporta il collateral globale al **50%**. Quello del secondo prestito resta al 45%.

Compensazione: riserva limitata e credito residuo

Il capitale insoluto è 0,5 ETH, ma la riserva contiene solo 0,025 ETH. Il contributore riceve **0,025 ETH direttamente nel wallet**; il credito residuo è **0,475 ETH**. Il tentativo al blocco **334** paga zero senza cancellare il credito né contare il fallimento una seconda volta.

Contabilità del pool (ETH)	Prima	Dopo / nuovo tentativo
Posizione totale del contributore	50,000	49,975
Capitale disponibile del contributore	49,500	49,500
Capitale bloccato del contributore	0,500	0,475
Riserva di compensazione	0,025	0
ETH effettivamente detenuti dal pool	49,525	49,500

Interpretazione. I 0,475 ETH bloccati sono un **credito non recuperato, non denaro prelevabile**. La compensazione riduce sia la posizione totale sia il capitale bloccato, lasciando invariato il disponibile: $49{,}975 - 0{,}475 = 49{,}5$ ETH. Reintegri della riserva o rimborsi tardivi possono consentire altri recuperi, ma non sono garantiti. **Zero prestiti attivi non significa zero perdite.**

Saldi dei wallet e numero di transazioni

Il wallet finale del contributore contiene **50,0499454037 ETH**, con il gas già considerato: vi sono arrivati guadagno e compensazione, mentre 49,5 ETH restano disponibili nel pool. Questa esecuzione aggiunge **11 transazioni della demo + 1 aggiornamento dell'oracolo + 2 voti YES = 14**; lo setup ne aggiunge sei (tre trasferimenti e tre distribuzioni). L'oracolo contiene 0,0000098852 ETH perché anche una precedente richiesta per un indirizzo inadeguato aveva pagato la commissione; quella coppia richiesta/risposta non rientra nelle 14 transazioni. Blocchi e saldi sono quelli dell'esecuzione osservata; riusare una distribuzione o cambiare dati può modificarli.

5. Test e valutazione del gas

5.1 Esecuzione e risultati dei test

Dopo lo setup delle dipendenze (pagina 3), lavorare dalla **radice del progetto**. **Non servono Geth, oracolo, voter, chiavi, configurazioni dei servizi o file Bitcoin reali.** Hardhat usa chain simulate isolate; Python usa dati temporanei e servizi simulati con mock. I test non modificano la distribuzione privata esistente. Non eseguire `python/setup.py` soltanto per fare i test.

Compilazione e suite complete

```
npm run typecheck
npm test
./.venv/bin/python -m unittest discover -s python/tests -v
```

Typecheck: compila i contratti ed esegue `tsc -noEmit`; basta l'uscita senza errori, anche se TypeScript non stampa messaggi. **Contratti:** l'esito atteso è **34 passing**, cioè **4 test Solidity + 30 Node.js/TypeScript**. Un test Solidity usa **256 input generati** mediante fuzzing: sono ripetizioni dello stesso test, non 256 test aggiuntivi. **Python:** attendersi **Ran 49 tests** e **OK**.

Gli scenari dei contratti coprono vincoli del voto e variazioni di saldo, soglie BTC, resti di allocazione, rate, ordine dei rimborsi, interessi, default, compensazioni ripetute, autorizzazioni, terminazione, aggiornamenti e reentrancy. Controlli espliciti verificano emittenti e parametri degli eventi e contratti indipendenti per due debitori. I test Python coprono parser, esclusione del genesis, UTXO, indicizzazione atomica, ripresa di eventi e transazioni, voter sempre favorevole, protezioni dello setup e compensazione della demo. I saldi simulati verificano la logica, non validano il dataset Bitcoin reale.

Eseguire un controllo mirato

```
npm test -- test/advanced_behaviors.test.ts
npm test -- test/events_and_isolation.test.ts
npm test -- test/reentrancy.test.ts
npm test -- test/upgrade.test.ts
npm test -- contracts/test/LoanContract.t.sol
./.venv/bin/python -m unittest discover -s python/tests \
  -p 'test_demo_scenarios.py' -v
```

Scegliere il comando per: contabilità; eventi e indipendenza dei prestiti; reentrancy; aggiornamenti; regole native del prestito; oppure **sei controlli offline della demo**. Gli eventi includono un nuovo tentativo di compensazione che non deve emettere un secondo default. Il test di indipendenza rimborsa due prestiti con debitori diversi e verifica che nessuno alteri condizioni, rimborsi o quote dell'altro.

Esaminare la copertura del codice

```
npm run test:coverage
```

Coverage: attendersi **34 passing** e percentuali di righe e istruzioni coperte. Aprire `coverage/html/index.html` nel browser. È copertura dei contratti, non Python né prova di sicurezza. Il bytecode strumentato altera il gas: questa modalità lascia intenzionalmente invariate le misure ordinarie. Comandi di profilazione e tabella sono a pagina 6; non usare il gas della coverage per valutare i costi.

Riconoscere il successo e risolvere i problemi

Un test può causare un revert e passare comunque: verifica il rifiuto di un'operazione non valida. Il test reentrancy passa solo se la variante insicura è svuotata *e* quella normale protegge i fondi onesti. Leggere il riepilogo finale: asserzioni fallite, `FAIL`/`ERROR` o uscita non nulla richiedono indagini. I conteggi descrivono questa versione e possono crescere con nuovi test.

Moduli o interprete mancanti: usare `./.venv/bin/python` e verificare cartella, versione Node e completamento di `./setup.sh`. **Test fallito:** rieseguire il file e leggere il primo scarto tra atteso e ottenuto, senza eliminare dati della chain o code. **Dati Bitcoin mancanti:** correggere il dataset per controllo e demo (pagina 3); i mock non lo validano. La demo reale è una prova di integrazione: non sostituisce i test automatici, né viceversa.

5. Test e valutazione del gas (continua)

5.2 Misurazione e interpretazione del gas

I test funzionali verificano il comportamento; la profilazione misura il costo delle operazioni. Entrambi usano lo stesso progetto Hardhat e gli stessi contratti, con bytecode ordinario per le misurazioni.

Ambiente. Hardhat 3.18.1, TypeScript 6.0.3, `node:test`, viem 2.57.3; Solidity 0.8.24, ottimizzatore con `runs=200`, EVM/hard-fork **Berlin**, chain ID simulato 202526. Lo scenario ha un solo contributore.

Riprodurre le misure

```
npm run test:gas
./.venv/bin/python docs/render_gas_table.py --language it
```

L'esito atteso è **1 passing**: un test di profilazione per **20 scenari**. Registra `gasUsed` delle ricevute, incluso il gas intrinseco, in `docs/gas-measurements.json`; il comando Python genera `docs/gas-table-it.tex`. Ricompilare questa relazione per aggiornare la tabella. Anche `npm test` include la profilazione e riscrive il JSON.

Costi del gas misurati

Operazione	Scenario	Unità di gas
FundingPool / tramite proxy		
`contribute()`	Nuovo contributore	140.923
`withdraw()`	Prelievo parziale	47.933
`withdraw()`	Prelievo finale	32.412
`createLoanProposal()`	Nuova proposta	234.798
`voteOnProposal()`	Voto favorevole	106.936
`executeApprovedLoan()`	Proposta approvata	1.049.274
`executeApprovedLoan()`	Proposta respinta	90.860
`claimInterestGain()`	Riscossione del guadagno	24.691
`requestCompensation()`	Compensazione pagata	52.652
`requestCompensation()`	Riserva vuota	43.786
`upgradeToAndCall()`	V2 compatibile	37.769
`terminate()`	Nessun prestito attivo	36.295
LoanContract		
`repay()`	Rata parziale	168.082
`repay()`	Rata finale	90.716
`repay()`	Rimborso in ritardo	138.782
`markDefaulted()`	Prestito scaduto	103.758
BitcoinOracle		
`requestUpdate()`	Richiesta a pagamento	25.193
`update()`	Primo saldo non nullo	49.426
`update()`	Saldo già presente	32.326
`withdrawFees()`	Riscossione delle commissioni	30.868

Leggere i valori e calibrare la commissione dell'oracolo

I valori sono **unità di gas**, non wei né il limite di gas inviato. La commissione in wei è **gas consumato per prezzo del gas**. Le letture tramite `eth_call` non richiedono una commissione di transazione.

Commissione dell'oracolo: $49\,426 \times 0{,}1$ gwei $= 4\,942\,600\,000\,000$ wei. La calibrazione usa il primo saldo non nullo di un indirizzo storico di 34 caratteri; aggiornare una voce esistente costa meno. Cambiare bytecode o impostazioni richiede una nuova calibrazione.

Validità delle misure. Sono risultati riproducibili di scenari specifici, non limiti massimi universali. Dipendono dallo stato dello storage e dal numero di contributori: il lavoro del pool può crescere con quest'ultimo. La coverage esegue gli stessi percorsi con bytecode strumentato senza sovrascrivere il JSON ordinario; per valutare i costi usare il comando di profilazione riportato sopra.